

TECHNIKAI ÁTTEKINTÉS

Grafika féléves beadandó – DY98R5

Ez a dokumentum a beadandó felépítését, működési logikáját és a követelményekhez való kapcsolódását foglalja össze. A célja annak a megmutatása, hogy a program milyen modulokra épül, ezek hogyan kapcsolódnak egymáshoz, és melyik követelmény hol, milyen megoldással teljesül.

1. A program célja és alapötlete

A beadandó egy bejárható, belső nézetes 3D jelenet, amely egy alállomás jellegű ipari környezetet jelenít meg. A felhasználó a billentyűzet és az egér segítségével mozoghat a térben, körbenézhet, interaktív elemeket jelölhet ki, majd kapcsolatba léphet velük. A program ezért nem pusztán egy statikus modellnézegető, hanem egyszerű játékméleti logikával rendelkező grafikai alkalmazás.

A központi játékméleti ötlet az, hogy a jelenetben található kapcsolókat meg kell javítani. Ezekhez lámpák tartoznak, a rendszer állapota ezek alapján változik, majd meghatározott feltételek teljesülése után a kapu kinyílik, és a végső terminál elérhetővé válik. A projekt tehát egyszerre mutat meg 3D megjelenítést, interakciót, állapotkezelést, animációt és egyszerű feladatlogikát.

2. A minimális követelmények teljesítése

A feladat kiírásában szereplő alapkövetelmények mindegyike egyértelműen megjelenik a programban, és külön modulokhoz köthető.

Kamerakezelés és a tér bejárása

A virtuális tér bejárását a kamera modul valósítja meg. A `main.c` kezeli az egér és a billentyűzet eseményeit, a `camera.c` pedig ezekből számolja ki a kamera új állapotát. A W, A, S, D billentyűk a kamera irányához képest mozgatják a játékost, az egér pedig a yaw és pitch szögeket módosítja. A nézet tényleges beállítása a `gluLookAt` használatával történik. Ez azért jó megoldás, mert a mozgás, a nézeti állapot és a renderelt kamera világosan elválik egymástól.

Külön fájlból betöltött térbeli objektumok

A jelenet objektumai külső fájlokból érkeznek. A `scene_loader.c` a `scene.csv` alapján olvassa be, hogy melyik objektum milyen modellt és textúrát használ, az OBJ modellek tényleges betöltését pedig a külső `app/ext/obj` könyvtár végzi. Ez megfelel annak a követelménynek, hogy a modellek ne a forráskódban legyenek beágyazva, hanem külön állományokból töltődjenek be.

Interaktivitás és animáció

A program interaktív, mert a játékos mozgása, a kijelölés, az E gombhoz kötött interakció és a világ állapotának változása mind valós időben történik. A kapu nyitása nem ugrásszerű, hanem időalapú animáció: a `scene_update.c` a `delta_time` alapján fokozatosan közelíti az aktuális szöget a célértékhez. Emellett a lámpák és a hibás kapcsolók vizuális pulzálást kapnak, a kapcsolóknál pedig szikraszerű effekt is megjelenik.

Textúrák használata

A modellek textúráit a texture.c tölti be BMP állományokból, majd a renderer ezeket az adott objektum rajzolásakor használja. A textúrázás külön modulba került, így a scene betöltése és a megjelenítés nem keveredik a képfeldolgozás technikai részleteivel. A textúrázott talaj külön kezelést is kap, hogy a felület ismétlődő mintázattal jelenjen meg.

Fények állítása

A programban a globális fényerő futás közben módosítható a + és - billentyűkkel. Az inputkezelés a main.c-ben történik, a tényleges OpenGL fényparaméterek frissítését pedig az app_render.c végzi. A megoldás nem egyszerű képernyő-sötétítés, hanem a fény komponenseinek újraállítása, ezért a jelenet megvilágítása valóban változik.

Használati útmutató megjelenítése

Az F1 billentyű megnyomására külön help overlay jelenik meg. Ennek betöltését és kirajzolását a help.c kezeli, a main.c pedig gondoskodik arról, hogy a help nézet közben a kamera ne forogjon tovább. Ez jól illeszkedik a kiírásban szereplő használati útmutató követelményhez.

Strukturált és következetes felépítés

A program szerkezete moduláris: külön kezeli a vezérlést, a kamerát, a jelenet betöltését, az állapotfrissítést, a geometriai lekérdezéseket, a renderelést és a 2D felületi elemeket. A repetitív pályaadatok kiszervezve kerültek a scene.csv fájlba, a főprogramban pedig minimális számú globális változó szerepel. Ez a felépítés közvetlenül támogatja az átláthatóságot és a védhetőséget.

3. TöbblETFunkciók és erősebb megoldások

A kötelező elemeken felül a beadandó több olyan megoldást is tartalmaz, amelyek a kiírás szerinti többlETFunkciók közé sorolhatók, és a projekt minőségét is javítják.

Köd és annak dinamikus állítása

A köd hatását az app_render.c valósítja meg, és futás közben a , illetve . billentyűkkel módosítható a sűrűsége. Ez nem csak látványelem, hanem segít a jelenet hangulatának kialakításában is.

Ütközésvizsgálat

A játékos mozgását AABB alapú ütközésvizsgálat korlátozza. A main.c a kamera helyzetéből játékosdobozt készít, a scene_query.c pedig megnézi, metszi-e valamelyik szilárd objektum határolódobozát. Így a programban nem lehet indokolatlanul átmenni a falakon.

Stencil buffer használata

A kijelölt objektum outline kiemelése stencil buffer segítségével történik a scene_render.c-ben. Ez látványos, ugyanakkor technikailag is indokolt megoldás, mert jól elkülöníti az interaktív elemeket a jelenet többi részétől.

Felhasználói felület

A célkereszt, az alsó információs sáv, az időmérés és a küldetés végét jelző overlay mind külön 2D felületi réteggként készültek el a ui_hud.c-ben. Ez a program használhatóságát és bemutathatóságát is javítja.

Objektumkijelölés

A program a kamera középpontjából indított nézeti sugárral választja ki az interaktív elemeket. Bár ez nem egérkattintásos picking, funkcionálisan ugyanazt a célt szolgálja: a jelenet tárgyai egyértelműen kijelölhetők, és az interakció ehhez kapcsolódik.

4. A projekt szerkezete és a főbb mappák szerepe

A projekt központi része az app mappa, mert itt található a beadandó tényleges forráskódja. Az include könyvtár a headereket, vagyis a publikus és belső interfészeket tartalmazza. Az src könyvtárban vannak azok a forrásállományok, amelyek a futási logikát, a renderelést és az interakciót megvalósítják.

Az assets mappa a jelenethez tartozó adatokat tartalmazza: modelleket, textúrákat és a scene.csv állományt. A pálya tehát adatvezérelt, vagyis az ismétlődő, adattárolásra emlékeztető részek nem a kódban vannak kézzel felsorolva, hanem külső leírófájlban. Ez közvetlenül megfelel a kiírás azon elvárásának, hogy az ilyen jellegű repetitív részek kerüljenek kiszervezésre.

Az app/ext/obj mappa egy külső, statikus segédkönyvtárat tartalmaz az OBJ modellek betöltésére és kirajzolására. Ez megfelel annak a követelménynek is, hogy külső forrásból származó modellbetöltő könyvtár licenzfeltüntetéssel és külön egységként szerepeljen a projektben.

A gyakorlatok mappa különálló, kisebb gyakorlóprogramokat tartalmaz. Ezek nem a féléves beadandó motor részét alkotják, hanem a korábbi SDL/OpenGL feladatok eredményei, ezért a beadandó értelmezése szempontjából az app és assets mappák tekinthetők elsődlegesnek.

5. A főbb fájlok szerepe

main.c

A program belépési pontja. Inicializálja az SDL-t és az OpenGL-t, létrehozza az ablakot, beállítja az alap renderállapotokat, betölti a scene-t, majd futtatja a fő ciklust. A main.c fontos tulajdonsága, hogy nem itt van az összes részletlogika, hanem innen történik a modulok összehangolása.

camera.c

A kamera pozícióját, forgatását és mozgását kezeli. A kamera update része az inputból számolja az elmozdulást, az apply rész pedig a nézetet állítja be. Ezzel a mozgás és a rendernézet logikailag is különválnak.

app_render.c

A globális renderkörnyezetért felel: perspektivikus vetítés, alapvilágítás, dinamikus lámpák, köd és debug rács. Itt vannak azok a megoldások, amelyek nem egyetlen objektumhoz, hanem a teljes jelenethez kapcsolódnak.

help.c és ui_hud.c

A 2D kezelőfelületet adják. A help.c az F1-re megjelenő súgót kezeli, a ui_hud.c pedig a célkeresztet, az alsó HUD elemeket, az időmérést és a győzelmi overlayt.

texture.c

A textúrák technikai kezelője. BMP fájlokat tölt be, ezekből OpenGL textúrát hoz létre, és biztosítja, hogy a renderer már előkészített textúrákkal dolgozhasson.

scene_loader.c

A jelenet felépítéséért felel. Beolvassa a scene.csv tartalmát, regisztrálja a modelleket és textúrákat, létrehozza az objektumokat, majd összerakja a Scene struktúrát.

scene_update.c

A jelenet állapotfrissítő modulja. Itt történik a kapcsolók, lámpák, kapu és interakciós logika összekapcsolása, valamint az időfüggő animációk számítása.

scene_query.c

A geometriai és logikai lekérdezések modulja. Ide tartozik az AABB számítás, az ütközésvizsgálat, a ray–AABB alapú picking és az aktív fényforrások adatainak lekérdezése.

scene_render.c és scene_effects.c

A 3D jelenet tényleges kirajzolását végzik. A render modul az objektumok transzformját és rajzolását kezeli, az effects modul pedig a speciális vizuális elemeket, például a glow hatást, a hibajelző fényeket és a szikrákat.

6. A program működési lánc futás közben

A futás fő logikája röviden így foglalható össze: inputkezelés, állapotfrissítés, geometriai lekérdezések, 3D renderelés, majd 2D kezelőfelület. Ez a sorrend azért előnyös, mert minden lépés a korábbiak eredményére épül.

A main.c először feldolgozza a billentyűzetet, az egeret és az ablakméret változását. Ezután kiszámolja az eltelt időt, majd meghívja a scene_update függvényt, amely a világ időfüggő állapotait frissíti. Ezután a kamera új pozíciója számolódik ki, majd ütközésvizsgálat történik, hogy a játékos ne kerülhessen tiltott helyre.

A következő fontos lépés a picking, vagyis annak meghatározása, hogy a játékos éppen melyik interaktív objektumra néz rá. Ha interakció történik, a scene_interact kezeli az eseményt. Ezután a program eldönti, hogy a küldetés feltételei teljesültek-e, végül kirajzolja a jelenetet és a HUD elemeket.

A fő ciklus felépítése tehát nem véletlenszerű: a program előbb adatot gyűjt, aztán állapotot frissít, majd ezek alapján dönt, végül megjeleníti az eredményt. Ez a szerkezet egyszerre átlátható és védhető.

7. A játékmeneti logika és az ok-okozati lánc

A beadandó játékmeneti logikája tudatosan egyszerű, de jól követhető láncra épül: kapcsoló, lámpa, kapu, terminál. Ez azt jelenti, hogy a világ állapotának változásai nem elszigetelten történnek, hanem egymásból következnek.

A játékos a jelenetben mozogva rá tud nézni interaktív elemekre. A kiválasztás a scene_pick segítségével történik. Ha a felhasználó megnyomja az interakciós gombot, a scene_interact kezeli az eseményt. A kapcsolók javítása a SceneEntity state mezőjének módosításával történik.

A lámpák állapota nem külön kézzel fenntartott listából származik, hanem a kapcsolók állapotából van levezetve. A program a kapcsoló és a lámpa azonosítójának végén szereplő szám alapján párosítja őket. Ez egyszerű, mégis jól karbantartható megoldás, mert nem kell külön kapcsolatlistát tárolni.

A kapu állapota szintén a világ állapotából következik. Ha elegendő számú kapcsoló aktív, a `scene_recompute_power` beállítja a kapu célállapotát. A kapu ezután nem ugrik azonnal új helyzetbe, hanem időfüggő animációval nyílik ki. A végső terminál csak akkor használható, ha a rendszer már helyreállt és a játékos a megfelelő helyre is eljutott. Ezért a befejezés egyszerre logikai és térbeli feltételhez kötött.

8. Tervezési döntések és azok indoklása

A program felépítésének legfontosabb célja az volt, hogy a megoldás ne egyetlen nagy, nehezen átlátható forrásfájlban legyen összekeverve. Emiatt külön modul kezeli a vezérlést, a kamerát, a scene betöltését, a state update részt, a geometriai lekérdezéseket, a renderelést és a 2D felületi elemeket.

A `scene.h` és `scene_internal.h` szétválasztása tudatos döntés. A publikus függvények és adattípusok elérhetők a külvilág számára, a belső struktúrák és segédfüggvények azonban a scene modulon belül maradnak. Ez C nyelven is lehetővé teszi egy OOP-szerűbb, jobban tagolt szerkezet kialakítását.

Az AABB alapú ütközésvizsgálat és picking azért jó választás, mert a jelenet méretéhez képest egyszerű, gyors és kellően pontos. Nem volt cél teljes fizikai motor vagy mesh-pontos metszésvizsgálat készítése; a választott megoldás a beadandó követelményeihez arányos és jól védhető.

A `scene.csv` használata azért különösen fontos, mert a repetitív pályaadatok így nem a kódba kerülnek bele. Ez a projekt szerkezetét tisztábbá teszi, és közvetlenül reagál a kiírásban szereplő elvárásra, miszerint az adattárolásra emlékeztető részeket célszerű külön adatfájlba kiszervezni.

9. Záró összegzés

A beadandó lényege nemcsak az, hogy egy 3D jelenet kirajzolódik, hanem az is, hogy a mozgás, az interakció, a világ állapota és a megjelenítés egymásra épülő rendszert alkot. A projekt adatvezérelt, moduláris és a kiírás követelményeihez igazított szerkezetet kapott.